

Sensores e Sinais

Introdução aos timers por hardware e conversores analógico-digital (ADC)

Luis Silva, Paulo Pedreiras, Pedro Fonseca (DETI/UA/IT)

2025-2026

Informações da aula

- Duração estimada: 2 horas em sala
- Tópico: Timers por Hardware, Aquisição de Sinais (ADC) e Análise Espectral
- Baseado em: documentação oficial (<https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c6/esp32-c6-devkitc-1/index.html#esp32-c6-devkitc-1>)
- Material e pré-requisitos
 - Hardware: ESP32-C6-DevKitC-1, cabo USB-C com suporte a dados, 1 LED (qualquer cor), 1 resistência 330Ohm, breadboard e fios de ligação
 - Software: Sistema operativo atualizado (sugerido Linux) e permissões de administrador

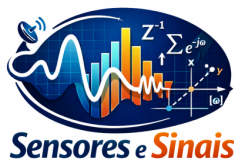
Objetivos

- Configurar temporizadores por hardware (*General Purpose Timers* - GPTimer)
- Utilizar a ADC (Analog-to-Digital Converter) do ESP32-C6 para amostrar sinais
- Validar experimentalmente os conceitos teóricos da Série/Transformada de Fourier aplicados a um sinal real

1. Conceitos essenciais (preparação prévia)

1.1. Timers por hardware – o que são

Nos exercícios do guião prático anterior, utilizámos funções como o `vTaskDelay()` para criar pausas na execução do nosso código e controlar a periodicidade da execução de determinadas operações da nossa aplicação (p. ex., alternar o estado de um GPIO a cada X



segundos). No entanto, essas funções possuem algumas limitações e desvantagens. Por exemplo:

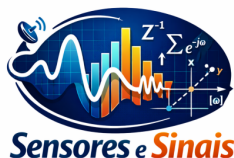
- Introduzem *overhead* no sistema operativo dado que exigem processamento para gerir a mudança de estado da tarefa (escalonamento) e monitorização de lista de tarefas adormecidas.
- A resolução está limitada pelo sistema operativo (tick rate) e é insuficiente para ordens temporais inferiores a milissegundos.
- Bloqueiam a tarefa que as invoca até o tempo pedido ter decorrido, não sendo possível realizar outras operações dentro da mesma tarefa.
- O uso de delays não garante a periodicidade de uma tarefa, apenas um intervalo de tempo entre o fim de uma tarefa (a anterior) e a próxima. No caso da amostragem (p. ex., `while(1) { amostra(); delay(x); }`), o tempo de execução da função `amostra()` soma-se ao atraso. Logo, nem a frequência de amostragem (f_s) desejada, nem o instante de cada amostra, são exatos e garantidos.

Uma outra alternativa para a contagem de tempo é o uso de temporizadores (timers) por hardware. Estes são periféricos físicos dentro do microcontrolador que, através do uso de vários sinais de relógio (clocks) dedicados, permitem medir intervalos temporais e gerar “alarmes” com alta resolução em “background” e de forma totalmente independente do processador principal.

1.1.1. Alarmes e Interrupções (ISR – Interrupt Service Routine)

Os temporizadores por hardware normalmente possuem a possibilidade de gerar **alarmes** sempre que a sua contagem temporal atinge um determinado valor definido pelo utilizador no momento da sua configuração. Quando um alarme é disparado, é gerada uma **interrupção**. Uma interrupção força o processador principal a parar imediatamente “o que está a fazer” (p. ex., a executar uma determinada instrução na função `app_main()` do nosso programa) para executar um pequeno bloco de código chamado **Interrupt Service Routine** (ISR). O bloco de código a ser executado aquando da interrupção gerada por um alarme de um determinado timer é especificado aquando da configuração do timer.

Nota: Dado que a ISR interrompe o funcionamento normal do microcontrolador, esta deverá ser o mais breve possível. Dentro do bloco de código associado nunca deverá colocar delays (p. ex. `vTaskDelay`), execução de algoritmos complexos e demorados, ou funções “lentas” como o `printf`. O objetivo deverá ser apenas realizar pequenas



operações (p. ex., atualizar uma variável ou alterar o estado de um GPIO) ou assinalar ao programa principal que algo ocorreu (p. ex. ativando uma variável booleana).

1.2. Configuração de timers por hardware no ESP32-C6

No ESP32-C6, usamos o driver **GPTimer** (Peripherals API) [1] para configurar timers por hardware. Em geral, são necessários os seguintes passos lógicos para configurar e utilizar um timer no nosso programa:

- I. **Instanciar e configurar o timer.** Inclui definir qual a resolução do contador em Hz (p. ex. 1 MHz para que cada incremento da contagem do timer corresponda a 1 microssegundo).
- II. **Configurar a ISR,** especificando qual a função C que deve ser executada quando o alarme disparar.
- III. **Configurar o alarme.** Inclui definir a contagem alvo (p. ex., disparar no valor de contagem 500 000), ativar a funcionalidade de reinício de contagem automático, e especificar o valor de contagem aquando reinício de contagem (p. ex., recomeçar no 0).
- IV. **Ativar e iniciar o timer.** Ligar o periférico e iniciar a contagem.

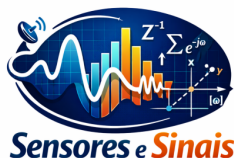
A Figura 1 apresenta o código de um pequeno programa que faz uso de um timer para gerar um alarme com periodicidade de 1 segundo. **Analise o código e identifique as principais funções e estruturas de dados utilizadas para configuração do timer.** A documentação oficial do driver GPTimer em [1] disponibiliza detalhes adicionais e mais exemplos.

```

1  #include <stdio.h>
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/task.h"
4  #include "driver/gptimer.h"
5  #include "esp_check.h" // Disponibilizar a macro ESP_ERROR_CHECK. Esta permite verificar o sucesso e error codes da execução de funções,
6  // terminando o programa e imprimindo informação para debug em caso de erros.
7
8  // Variável global para permitir o seu acesso pela ISR e pelo programa principal.
9  // 'volatile' é necessária para avisar o compilador que esta pode mudar a qualquer momento (pela ISR)
10 // e impedir que este a optimize incorrectamente (p. ex. faça caching do seu valor).
11 volatile uint32_t my_counter = 0;
12
13 // Função de callback do alarme (ISR - Interrupt Service Routine).
14 // Esta é a assinatura comum para a callback, com parâmetros que permitem receber informação de contexto.
15 static bool timer_isr_callback(gptimer_handle_t timer, const gptimer_alarm_event_data_t *edata, void *user_data) {
16
17     my_counter++;
18
19     // Valor lógico retornado serve para controlar alguns aspectos de gestão de tarefas do FreeRTOS.
20     // Não vamos entrar nesses detalhes nesta altura; vamos retornar sempre "false".
21     return false;
22 }
23
24 void app_main(void) {
25
26     printf("Starting timer demo...\n");
27
28     // I) Instanciar e configurar o temporizador (1 MHz de resolução).
29     gptimer_handle_t my_timer = NULL;
30     gptimer_config_t timer_config = {
31         .clk_src = GPTIMER_CLK_SRC_DEFAULT, // Seleciona uso do clock "default".
32         .direction = GPTIMER_COUNT_UP, // Configura contagem crescente.
33         .resolution_hz = 1000000, // Configura 1 MHz resolução. 1 unidade contagem (tick) = 1 microssegundo.
34     };
35     ESP_ERROR_CHECK(gptimer_new_timer(&timer_config, &my_timer));
36
37     // II) Configurar a ISR
38     gptimer_event_callbacks_t cbs = {
39         .on_alarm = timer_isr_callback, // Função "timer_isr_callback" deve ser invocada quando ocorrer um evento alarme.
40     };
41     ESP_ERROR_CHECK(gptimer_register_event_callbacks(my_timer, &cbs, NULL)); // Registo da ISR especificada anteriormente no nosso timer.
42
43     // III) Configurar o alarme
44     gptimer_alarm_config_t alarm_config = {
45         .alarm_count = 1000000, // O alarme dispara aos 1.000.000 tiques (1 segundo exato).
46         .reload_count = 0, // Contagem recomeça no tique 0.
47         .flags.auto_reload_on_alarm = true, // Recomeça contagem automaticamente após alarme.
48     };
49     ESP_ERROR_CHECK(gptimer_set_alarm_action(my_timer, &alarm_config));
50
51     // IV) Ativar e iniciar o timer
52     ESP_ERROR_CHECK(gptimer_enable(my_timer)); // Ativar periférico timer.
53     ESP_ERROR_CHECK(gptimer_start(my_timer)); // Iniciar contagem.
54
55     while (1) {
56         printf("Value of my counter: %lu\n", my_counter);
57         // Pausa de aproximadamente 1 segundo para não "inundar" o terminal com mensagens.
58         vTaskDelay(1000 / portTICK_PERIOD_MS);
59     }
60 }

```

Figura 1- Exemplo de uso de um timer por hardware no ESP32-C6



1.3. Analog-to-Digital Converter (ADC) – o que é

Como o nome indica, um ADC é na sua essência um circuito ou dispositivo que nos permite converter um sinal analógico, por exemplo a variação da tensão elétrica aos terminais de um sensor de temperatura ou microfone, numa representação digital do sinal.

Muitos microcontroladores possuem um único **módulo ADC** com **múltiplos canais** (**channels**), cada um deles ligado a diferentes pinos de entrada analógica do microcontrolador ou a outros circuitos internos (p. ex. fontes de tensão internas). Cada canal permite medir a ADC medir diferentes sinais, contudo, normalmente apenas é possível converter um canal de cada vez, sendo a conversão de múltiplos canais feita sequencialmente usando *multiplexing*.

O processo de conversão analógico-digital consiste de forma genérica nas seguintes etapas (Figura 2):

- I. **Amostragem.** A primeira etapa consiste na **amostragem do sinal analógico** em intervalos de tempo específicos. O **valor lido** num determinado **instante** é mantido “**constante**” até que a **conversão tenha terminado** (sample-and-hold).
- II. **Quantização.** Cada **valor lido** é **depois mapeado** para um nível discreto finito que a ADC pode representar. Em suma, a ADC divide uma amplitude máxima, o input range da ADC, numa série de níveis discretos igualmente espaçados. O valor lido é comparado com estes níveis e é-lhe atribuído o nível mais próximo. O “tamanho” de cada nível representa a resolução da ADC que pode ser calculado por $\Delta V = \frac{V_{ref}}{2^N}$, onde **Vref** é a **tensão de referência** que define o input range da ADC e **N** o **número de bits** da ADC. **O input range da ADC deverá ser superior à amplitude máxima esperada do sinal que pretendemos converter.** Caso a amplitude máxima exceda este valor, é possível configurar **atenuadores** para ajustar o input range da ADC.
- III. **Codificação (Digital Encoding).** Etapa onde cada valor *quantizado* é convertido para um código binário que pode depois ser processado pelos microcontroladores e outros sistemas digitais.

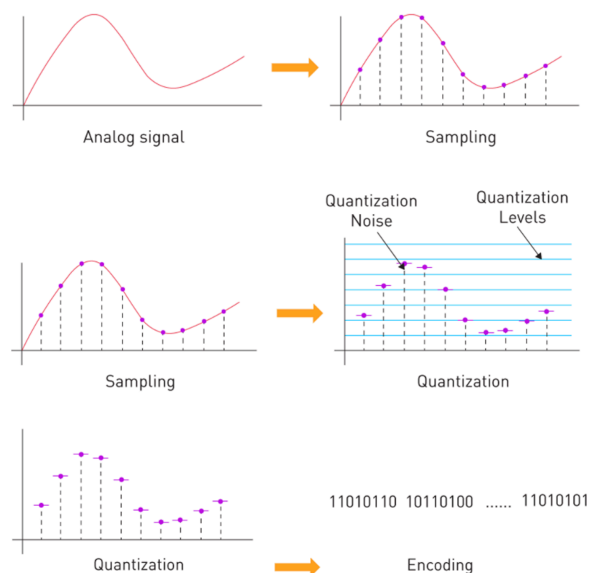


Figura 2 - Etapas principais da conversão analógico-digital (retirado de [2])

1.4. Configuração de ADCs no ESP32-C6

O ESP32-C6 possui um ADC de **12 bits**, com **sete canais** e um input range (**Vref**) de **1.1V** (0 a 1.1V). São suportados **4 níveis de atenuação** (0, 2.5, 6 e 12dB) sendo que o nível máximo permite obter um input range de **3.3V** (0 a 3.3V). Para mais detalhes, consultar o manual técnico do ESP32-C6 (disponível em [3]).

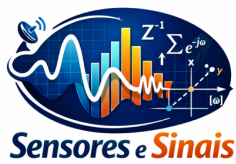
A ADC do ESP32-C6 suporta dois modos de operação distintos: (I) One-Shot Mode e (II) Continuous Mode [4].

1.4.1. One-Shot Mode

Permite obter uma única amostra de um determinado canal após ordem “manual” de leitura (on-demand). É adequado para aplicações que requerem uma amostragem infrequente ou apenas em determinados eventos.

Para operar a ADC neste modo, é utilizado o driver **adc_oneshot** [5] da API *Peripherals*. Em geral, são necessários os seguintes passos lógicos para configurar e operar a ADC neste modo no nosso programa:

- I. **Instanciar e configurar o driver.** Inclui especificar que módulo ADC usar, clock, e outras configurações.
- II. **Configurar o canal.** Inclui definir a resolução e atenuação. **Não é necessária qualquer configuração direta ao GPIO correspondente (p.ex. configurar**



este como entrada). A função de configuração do canal realiza as configurações do GPIO necessárias automaticamente.

- III. **Efetuar a leitura manual.** Dar a ordem de conversão ao módulo ADC, aguardar que a conversão termine, e guardar o valor obtido numa variável.

A Figura 1 apresenta o código de um pequeno programa que faz uso de uma ADC em modo one-shot para obter uma amostra de um sinal a cada 0.5 segundos (aproximadamente). **Análise o código e identifique as principais funções e estruturas de dados utilizadas para configuração da ADC.** A documentação oficial do driver `adc_oneshot` [5] disponibiliza detalhes adicionais e mais exemplos.

```

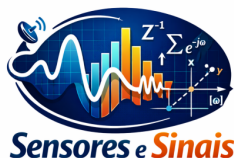
1  #include <stdio.h>
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/task.h"
4  #include "esp_adc/adc_oneshot.h"
5  #include "esp_log.h"           // Acesso à lib de logging. Permite usar ESP_LOGI com funcionalidades mais avançadas que o printf, como definir
6                                  // níveis de "severidade", timestamps, tags, e filtragem por nível de "severidade", etc.
7                                  // https://docs.espressif.com/projects/esp-idf/en/latest/esp32c6/api-reference/system/log.html
8
9  #define ADC_INPUT ADC_CHANNEL_3
10
11 // Etiqueta para os logs no terminal
12 static const char *TAG = "ADC_EXEMPLO";
13
14 void app_main(void) {
15     ESP_LOGI(TAG, "Starting adc one-shot mode demo...");
16
17     // I) Instanciar e configurar módulo ADC
18     adc_oneshot_unit_handle_t adc1_handle;
19     adc_oneshot_unit_init_cfg_t init_config = {
20         .unit_id = ADC_UNIT_1,           // Usar o módulo ADC 1
21         .clk_src = ADC_DIGI_CLK_SRC_DEFAULT, // Usar o default clock
22         .ulp_mode = ADC_ULP_MODE_DISABLE, // Desactivar modo de ultra-baixo consumo
23     };
24     ESP_ERROR_CHECK(adc_oneshot_new_unit(&init_config, &adc1_handle));
25
26     // II) Configurar o canal da ADC
27     adc_oneshot_chan_cfg_t config_canal = {
28         .bitwidth = ADC_BITWIDTH_12,     // Resolução padrão de 12 bits (níveis de 0 a 4095)
29         .atten = ADC_ATTEN_DB_12,        // Atenuação máxima (12dB) para conseguir ler tensões até ~3.9V
30     };
31
32     ESP_ERROR_CHECK(adc_oneshot_config_channel(adc1_handle, ADC_INPUT, &config_canal));
33
34     int raw_value = 0;
35     float value_in_volts = 0.0;
36     ESP_LOGI(TAG, "A iniciar leituras a cada 0.5 segundos...");
37
38     while (1) {
39         // III) Efetua a leitura do canal e guarda em 'valor_lido_raw'
40         esp_err_t erro = adc_oneshot_read(adc1_handle, ADC_INPUT, &raw_value);
41
42         if (erro == ESP_OK) {
43             // Converte o valor digital bruto (0-4095) para Volts (0.0 - 3.9)
44             // Nota: 0 cast (float) é crucial para evitar que o C faça uma divisão inteira e corte os decimais!
45             value_in_volts = ((float)raw_value / 4095.0) * 3.9;
46
47             // Imprime o valor bruto e o valor convertido (com 2 casas decimais)
48             ESP_LOGI(TAG, "Amostra: %d -> Tensao: %.2f V\n", raw_value, value_in_volts);
49         } else {
50             ESP_LOGE(TAG, "Falha ao ler a ADC!");
51         }
52
53         // Pausa exata de 0.5 segundos (fs = 2 Hz)
54         vTaskDelay(pdMS_TO_TICKS(500));
55     }
56 }

```

Figura 3 - Programa exemplo do uso de ADC em modo one-shot

1.4.2. Continuous Mode

Neste modo, a ADC é configurada para ler milhares de amostras por segundo de forma autónoma, guardando os resultados em memória e organizados numa estrutura de dados denominada **conversion frame**. É um modo útil para aplicações que requerem amostragens periódicas ou a uma frequência muito elevada.



O número de amostras que uma **conversion frame** contém é configurável. Cada amostra consiste em múltiplos bytes, sendo número de bytes fixo e definido pela macro **SOC_ADC_DIGI_RESULT_BYTES**.

A obtenção de uma **conversion frame** pode ser realizada por dois métodos: (I) através de um pedido bloqueante que aguarda até esta estar completa, (II) através do registo de uma ISR que é invocada aquando de uma interrupção gerada pela ADC quando completa a *conversion frame*.

Para operar a ADC neste modo, é utilizado o driver **adc_continuous** [6] da API *Peripherals*. Em geral, são necessários os seguintes passos lógicos para configurar e operar a ADC neste modo no nosso programa:

- I. **Instanciar e configurar driver.** Implica definir tamanho das *conversion frames* e buffer interno do driver para armazenar estas.
- II. **Configurar canal e módulo ADC.** À semelhança do one-shot, temos de configurar diversos parâmetros relativos à amostragem no canal e ao módulo ADC. Aqui também se poderá definir a IRS a ser invocada quando uma nova *conversion frame* está disponível para transferência.
- III. **Iniciar amostragem.** Dizer ao módulo ADC para começar o processo de amostragem em *background*.
- IV. **Transferir e processar.** Consoante o método usado (função bloqueante ou ISR), extrair as amostras da frame transferida por DMA e processar estas.

As figuras seguintes apresentam o código de um pequeno programa que faz uso de uma ADC em *continuous mode* para amostrar um canal a 20.000 Hz durante 1 segundo. Analise o código e identifique as principais funções e estruturas de dados utilizadas para configuração da ADC. A documentação oficial do driver **adc_continuous** [6] disponibiliza detalhes adicionais e mais exemplos.

```

1  #include <stdio.h>
2  #include <string.h>
3  #include "freertos/FreeRTOS.h"
4  #include "esp_adc/adc_continuous.h"
5  #include "esp_log.h"
6
7  static const char *TAG = "ADC_DMA";
8
9  #define SAMPLING_FREQUENCY_HZ 20000 // fs = 20 kHz. Existem valores mínimos e máximos suportados pela ADC, definidos
10 // pelas macro SOC_ADC_SAMPLE_FREQ_THRES_LOW e SOC_ADC_SAMPLE_FREQ_THRES_HIGH respetivamente. Macros definidas em "soc/soc_caps.h".
11 #define SAMPLING_OBSERVATION_TIME_SECONDS 1 // X: Tempo de observação para garantir resolução de 1 Hz.
12 #define SAMPLES_NUMBER_GOAL (SAMPLING_FREQUENCY_HZ * SAMPLING_OBSERVATION_TIME_SECONDS) // Número exato de amostras necessárias (N = fs * X).
13
14 // Vamos definir conversion frames com 64 amostras que resulta em cerca de 64 * 4 = 256 Bytes (SOC_ADC_DIGI_RESULT_BYTES no nosso caso é 4 Bytes).
15 // Temos assim um equilíbrio entre recursos necessários e número de vezes que o CPU é interrompido para a operação de leitura da frame (p. ex., caso usemos ISR).
16 #define CONVERSION_FRAME_LENGTH (64 * SOC_ADC_DIGI_RESULT_BYTES)
17
18 // !! A stack FreeRTOS para app_main é reduzida (4 a 8 KB). A declaração de variáveis "gigantes", como o array seguinte, deve ser feita aqui para ser alocada em RAM,
19 // caso contrário, vamos ter stack overflow e o programa "morre".
20 float processed_samples[SAMPLES_NUMBER_GOAL]; // Para guardar as nossas amostras processadas (p. ex., para depois usarmos no Octave).
21 uint32_t number_processed_samples = 0; // Contador com o número de amostras processadas e guardadas no nosso buffer principal.
22
23 void app_main(void) {
24     ESP_LOGI(TAG, "Starting adc continuous mode demo...");
25
26     // I) Instanciar e configurar driver.
27     adc_continuous_handle_t adc_handle = NULL;
28     adc_continuous_handle_cfg_t adc_handle_config = {
29         .max_store_buf_size = 4 * CONVERSION_FRAME_LENGTH, // Tamanho do buffer interno do driver para guardar amostras. // Deverá ser no mínimo o dobro de CONVERSION_FRAME_LENGTH.
30         .conv_frame_size = CONVERSION_FRAME_LENGTH, // Tamanho de cada conversion frame.
31     };
32     ESP_ERROR_CHECK(adc_continuous_new_handle(&adc_handle_config, &adc_handle));
33
34     // II) Configurar canal e módulo ADC.
35     adc_digi_pattern_config_t channel_config = {
36         .channel = ADC_CHANNEL_3, // Vamos amostrar o Canal 3 (GPIO 3 no ESP32-C6).
37         .atten = ADC_ATTEN_DB_12, // Atenuação máxima (12dB) para conseguir ler tensões até ~3.9V.
38         .unit = ADC_UNIT_1, // Usar o módulo ADC 1.
39         .bit_width = ADC_BITWIDTH_12, // Resolução padrão de 12 bits (níveis de 0 a 4095). !! Nota para evitar confusão: pode reparar que este campo neste driver é ".bit_width"
40         // e não ".bitwidth" como no driver one-shot. O porquê desta inconsistência...não sabemos.
41     };
42     adc_continuous_config_t adc_config = {
43         .pattern_num = 1, // Apenas um canal vai ser usado.
44         .adc_pattern = &channel_config, // Lista com a configuração para cada um dos canais usados.
45         .sample_freq_hz = SAMPLING_FREQUENCY_HZ, // Frequência de amostragem esperada.
46         .conv_mode = ADC_CONV_SINGLE_UNIT_1, // Apenas usar o módulo ADC 1 para conversão.
47         .format = ADC_DIGI_OUTPUT_FORMAT_TYPE2, // Existem dois formatos de estruturas de dados para armazenar o valor de cada
48         // amostra e metadata associada: ADC_DIGI_OUTPUT_FORMAT_TYPE1 e ADC_DIGI_OUTPUT_FORMAT_TYPE2.
49         // 0 formato padrão para o ESP32-C6 (e outros recentes) é ADC_DIGI_OUTPUT_FORMAT_TYPE2.
50     };
51     ESP_ERROR_CHECK(adc_continuous_config(adc_handle, &adc_config));
52
53     uint8_t dma_buffer[CONVERSION_FRAME_LENGTH]; // Buffer para a transferência de uma conversion frame da ADC para o nosso programa via DMA.
54     uint32_t number_read_bytes = 0; // Variável que irá indicar quantos bytes foram transferidos via DMA.
55
56     // III) Iniciar a amostragem.
57     ESP_ERROR_CHECK(adc_continuous_start(adc_handle));

```

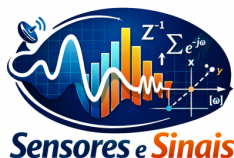
Figura 4 - Programa exemplo do uso de ADC em modo continuous (I)

```

56 // III) Iniciar a amostragem.
57 ESP_ERROR_CHECK(adc_continuous_start(adc_handle));
58
59 // IV) Transferir e processar conversion frames
60 while (1) {
61     // Pedir uma conversion frame completa via DMA. A função bloqueia até os dados estarem prontos.
62     esp_err_t erro = adc_continuous_read(adc_handle, dma_buffer, CONVERSION_FRAME_LENGTH, &number_read_bytes, portMAX_DELAY);
63
64     if (erro == ESP_OK) {
65         // Pode acontecer por vezes o número de amostras na frame ser inferior ao pedido (frame "incompleta").
66         // É boa prática calcular sempre quantas amostras efetivamente estão presentes na frame obtida.
67         uint32_t frame_samples_number = number_read_bytes / SOC_ADC_DIGI_RESULT_BYTES;
68
69         // Extrair, converter e guardar cada amostra obtida no nosso buffer principal.
70         for (int sample_index = 0; sample_index < frame_samples_number; sample_index++) {
71
72             // Se já atingimos a nossa meta de tempo (X segundos), paramos de guardar
73             if (number_processed_samples >= SAMPLES_NUMBER_GOAL) {
74                 break;
75             }
76
77             // Extrair a amostra usando a estrutura oficial da Espressif.
78             adc_digi_output_data_t *dados = (adc_digi_output_data_t*)&dma_buffer[sample_index * SOC_ADC_DIGI_RESULT_BYTES];
79             uint32_t raw_sample = dados->type2.data;
80
81             // Converter para Volts e guardar no nosso buffer principal.
82             processed_samples[number_processed_samples] = ((float)raw_sample / 4095.0) * 3.9;
83
84             number_processed_samples++;
85         }
86
87         if (number_processed_samples >= SAMPLES_NUMBER_GOAL) {
88             ESP_LOGI(TAG, "Desired number of samples achieved! Stopping sampling. First 10 collected samples:\n");
89             ESP_ERROR_CHECK(adc_continuous_stop(adc_handle));
90             for(int sample_index = 0; sample_index < 10; sample_index++) {
91                 ESP_LOGI(TAG, "Sample %d: %.2f V\n", sample_index, processed_samples[sample_index]);
92             }
93             break;
94         }
95     }
96 }
97 }

```

Figura 5 - Programa exemplo do uso de ADC em modo continuous (II)



2. Exercícios práticos

2.1. Geração de um sinal onda quadrada

Objetivo: Configurar um pino como saída digital (output) e manipular o seu estado lógico ao longo do tempo com recurso a um timer por hardware para gerar um sinal de onda quadrada.

Material necessário:

- 1 LED (qualquer cor)
- 1 Resistência de 330 Ohm
- Fios de ligação e breadboard
- Placa ESP32-C6-DevKitC-1 e cabo USB-C

Desenvolva um programa que seja capaz de gerar um **sinal de onda quadrada** no **GPIO 11** com **período 1s**. Para tal, deverá configurar um **timer** por hardware (GPTimer) para **gerar alarmes** com a **periodicidade adequada**. Deverá também **implementar uma ISR** que **manipule o nível lógico do GPIO 11** de modo a **gerar o sinal desejado**.

De modo a **visualizar a alternância do sinal**, **ligue um LED** (e resistência) ao **GPIO 11**. Confirme visualmente a correta frequência do sinal.

Finalmente, faça as alterações necessárias no código para que o sinal passe a ter uma frequência de **50 Hz**. Observe o efeito visual no LED e interprete a diferença com o cenário anterior.

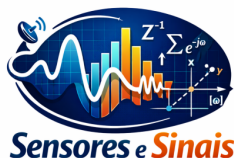
2.2. Amostragem de sinais com ADC em *continuous mode*

Objetivo: Configurar uma ADC em *continuous mode* com os parâmetros necessários para amostrar um sinal num GPIO.

Material necessário:

- Circuito do exercício 2.1 (GPIO 11 a gerar onda de 50 Hz).
- 1 fio de ligação extra

Desenvolva um programa que mantenha a geração da **onda quadrada de 50 Hz** (exercício 2.1) e que **utilize o driver `adc_continuous`** para amostrar esse mesmo sinal



usando o **canal 3 (ADC1_CH3)**. Para obter o sinal da onda quadrada no canal da ADC, ligue o GPIO do canal da ADC ao **GPIO 11** com um fio de ligação.

A amostragem deverá ser feita a uma frequência $f_s = 20.000\text{Hz}$ e com resolução espectral $\Delta f \leq 5\text{ Hz}$. Após recolha das amostras necessárias, apresente as primeiras **25 amostras** convertidas para valores de tensão (mV). Observe o resultado e interprete o fenómeno considerando a frequência do sinal original e da frequência de amostragem.

Notas:

- Algumas funções da biblioteca de processamento de sinal ESP-DSP [7] requerem que o número total de amostras seja uma potência de 2 (p. exx 512, 1024, 2048, 4096...). **Tendo em conta a resolução espectral exigida, calcule o número ideal de amostras e escolha a potência de 2 mais adequada.**

2.3. Análise espectral com Transformada Rápida de Fourier (FFT)

Objetivo: Realizar a FFT de um sinal amostrado usando a biblioteca oficial de processamento de sinal da Espressif ESP-DSP [7].

Material necessário:

- Circuito do exercício 2.2 (amostragem da onda de 50 Hz no GPIO 11).

No exercício 2.2, recolhemos amostras de uma onda quadrada de 50Hz. Esta não é composta por uma única frequência, mas pela soma de uma componente contínua (DC) com infinitas ondas sinusoidais de frequências múltiplas (harmónicos).

Neste exercício, altere o programa desenvolvido anteriormente para que, com recurso à biblioteca ESP-DSP e às suas funções para cálculo da FFT, este faça a **análise espectral das amostras recolhidas** e mostre no terminal as componentes cuja **magnitude normalizada seja superior a 0.200 V**. Deverá conseguir confirmar um “pico” nos 0 Hz (componente DC), 50 Hz (frequência fundamental do sinal), bem como nos harmónicos ímpares 150 Hz e 250 Hz.

Os passos lógicos para resolver este problema são:

- I. **Prepare as amostras para o domínio complexo.** A função FFT exige que os dados sejam número complexos. Crie um array de **floats** com o **dobro do tamanho das amostras recolhidas** e **preencha as posições pares com a parte**

- real** (as nossas amostras) e as ímpares com a parte imaginária (zeros no nosso caso).
- II. **Faça a inicialização da biblioteca com a função `dsps_fft2r_init_fc32(NULL, CONFIG_DSP_MAX_FFT_SIZE);`**
 - III. **Calcule a FFT.** Para tal, use a função `dsps_fft2r_fc32`.
 - IV. **Calcule a magnitude dos resultados e normalize.** Faça apenas considerando as frequências positivas.
 - V. **Filtre e mostre no terminal as componentes espectrais cuja magnitude seja superior a 0.200 V.** Indique a frequência correspondente para cada uma destas.

Notas:

- A biblioteca ESP-DSP antes de poder ser utilizada no nosso programa (`#include "esp_dsp.h"`), tem de ser adicionada ao nosso projeto através do ESP IDF Component Manager [8]. Para tal, abra um terminal ESP-IDF (p. ex. Figura 6) e execute o seguinte comando: `idf.py fullclean && idf.py add-dependency "espressif/esp-dsp"`. Após o sucesso destes comandos, deverá ser possível importar a biblioteca e fazer *build* do projeto sem erros.
- A função `dsps_fft2r_fc32` “baralha” a ordem dos resultados. Deverá executar a função `dsps_bit_rev_fc32` imediatamente após para reordenar o array corretamente.

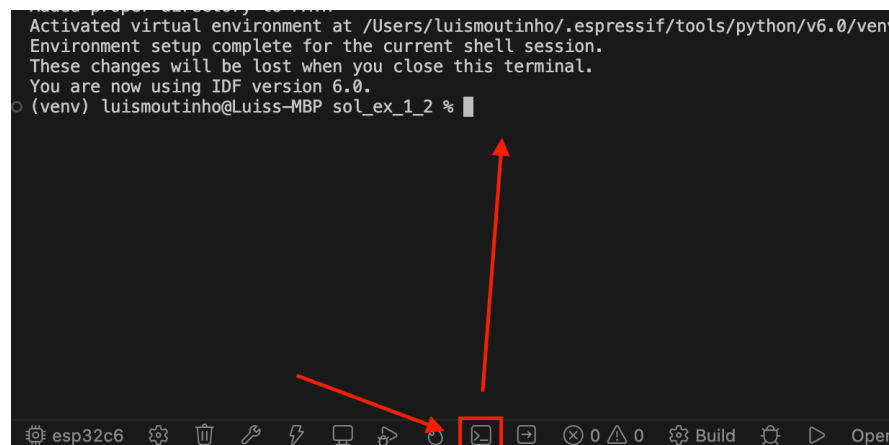
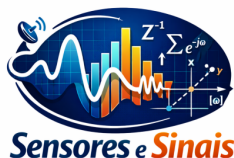


Figura 6 - Exemplo de como abrir terminal ESP-IDF no VS Code

3. Desafios extra (fora da sala)

3.1. Onda quadrada com seleção de frequência através de botão



Modifique o código do Exercício 2.1 e adicione a lógica necessária para que, cada vez que o botão **BOOT** seja pressionado, a frequência do sinal quadrado gerado se altere ciclicamente entre três valores (1Hz, 25Hz e 50Hz).

3.2. Análise espectral com display em gráfico

Modifique o programa desenvolvido no Exercício 2.3 para que mostre todas as componentes espectrais com magnitude superior a 0.1V e a sua frequência respectiva em forma de gráfico ASCII. Para tal, pode basear-se no programa exemplo oficial em [9].

Referências

- [1] <https://docs.espressif.com/projects/esp-idf/en/latest/esp32c6/api-reference/peripherals/gptimer.html>
- [2] <https://www.monolithicpower.com/en/learning/resources/comprehensive-guide-to-analog-to-digital-converters>
- [3] https://documentation.espressif.com/esp32c6_technical_reference_manual_en.pdf#sensor
- [4] <https://docs.espressif.com/projects/esp-idf/en/latest/esp32c6/api-reference/peripherals/adc/index.html>
- [5] https://docs.espressif.com/projects/esp-idf/en/latest/esp32c6/api-reference/peripherals/adc/adc_one-shot.html#introduction
- [6] https://docs.espressif.com/projects/esp-idf/en/latest/esp32c6/api-reference/peripherals/adc/adc_continuous.html
- [7] <https://docs.espressif.com/projects/esp-dsp/en/latest/esp32/esp-dsp-examples.html#fft>
- [8] <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/tools/idf-component-manager.html>
- [9] https://github.com/espressif/esp-dsp/blob/3c12d059f96ec95ed00769fb32d41ca23e20cdf1/examples/fft/main/dsps_fft_main.c